

EXPERIMENT 5 : 8-PUZZLE PROBLEM

Aim

To develop a Python program to solve the **8-Puzzle Problem** using **Artificial Intelligence search techniques**.

Objective

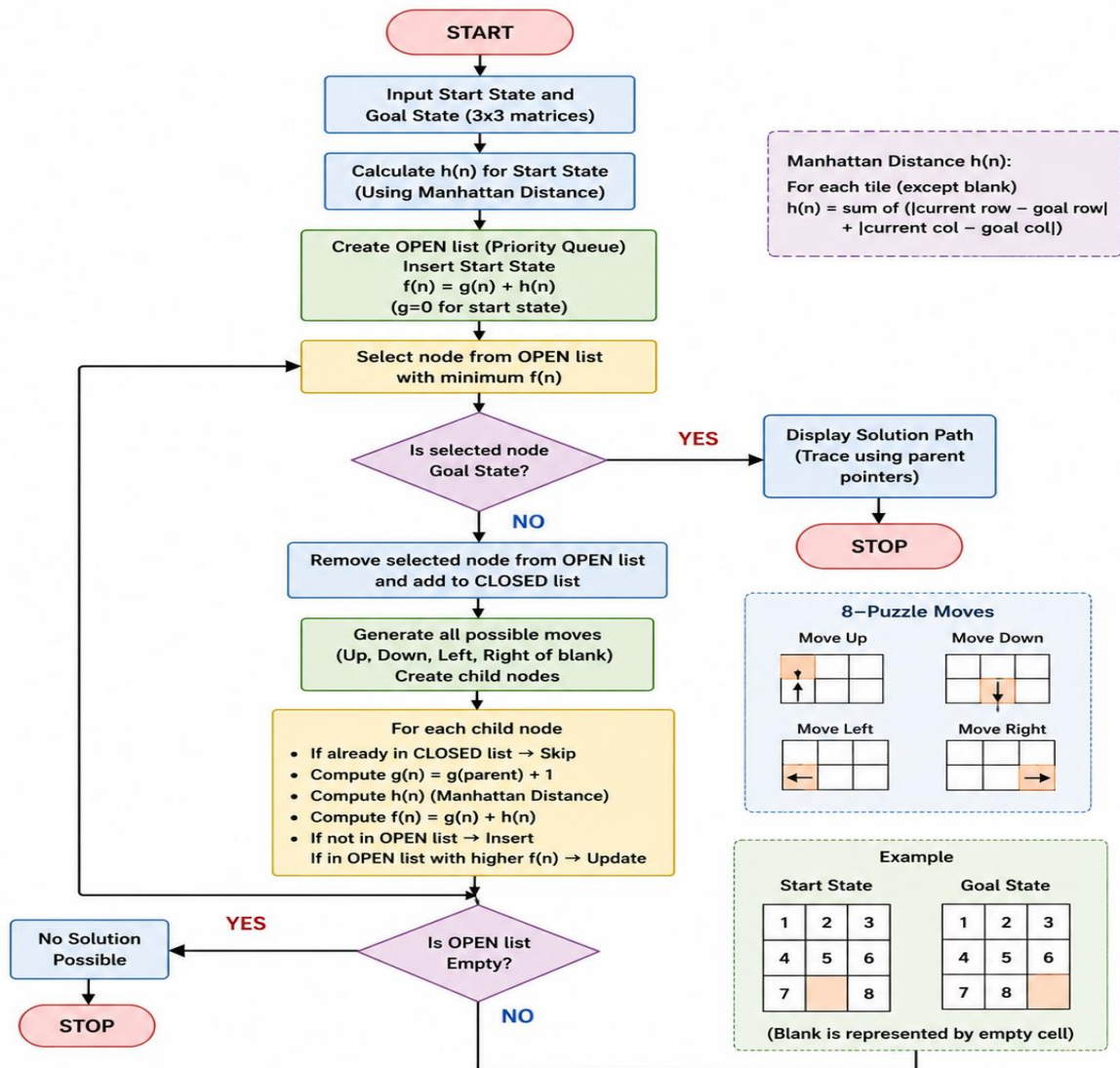
- To understand the concept of **state-space search** in AI.
- To represent the **8-puzzle** as a search problem.
- To implement a search algorithm to reach the **goal state**.
- To learn how an AI agent explores possible moves.

Algorithm

- **Start the program.**
- **Define the initial state** of the puzzle.
- **Define the goal state.**
- Find the position of the empty tile (0).
- Generate all possible moves:
 - Move Up
 - Move Down
 - Move Left
 - Move Right
- Create new puzzle states after each valid move.
- Compare the current state with the goal state.
- If the goal state is reached, display the solution.
- Otherwise, continue exploring the next possible states.
- **Stop** when the goal is found.

Flowchart:

8-PUZZLE PROBLEM – FLOWCHART



code:

from collections import deque

Goal state

```
goal = [[1, 2, 3],
        [4, 5, 6],
        [7, 8, 0]]
```

Initial state

```
initial = [[1, 2, 3],
```

```
[4, 0, 5],
```

```
[6, 7, 8]]
```

```
# Find position of 0
```

```
def find_zero(state):
```

```
    for i in range(3):
```

```
        for j in range(3):
```

```
            if state[i][j] == 0:
```

```
                return i, j
```

```
# Generate possible moves
```

```
def get_neighbors(state):
```

```
    neighbors = []
```

```
    x, y = find_zero(state)
```

```
    moves = [(-1,0), (1,0), (0,-1), (0,1)]
```

```
    for dx, dy in moves:
```

```
        nx, ny = x + dx, y + dy
```

```
        if 0 <= nx < 3 and 0 <= ny < 3:
```

```
            new_state = [row[:] for row in state]
```

```
            new_state[x][y], new_state[nx][ny] = new_state[nx][ny], new_state[x][y]
```

```
            neighbors.append(new_state)
```

```
    return neighbors
```

```
# Convert state to tuple for hashing
```

```
def state_to_tuple(state):
```

```
    return tuple(tuple(row) for row in state)
```

```
# BFS algorithm

def solve_puzzle(initial, goal):
    queue = deque([initial])
    visited = set()
    visited.add(state_to_tuple(initial))

    while queue:
        current = queue.popleft()

        print("Current State:")
        for row in current:
            print(row)
        print()

        if current == goal:
            print("Goal State Reached!")
            return

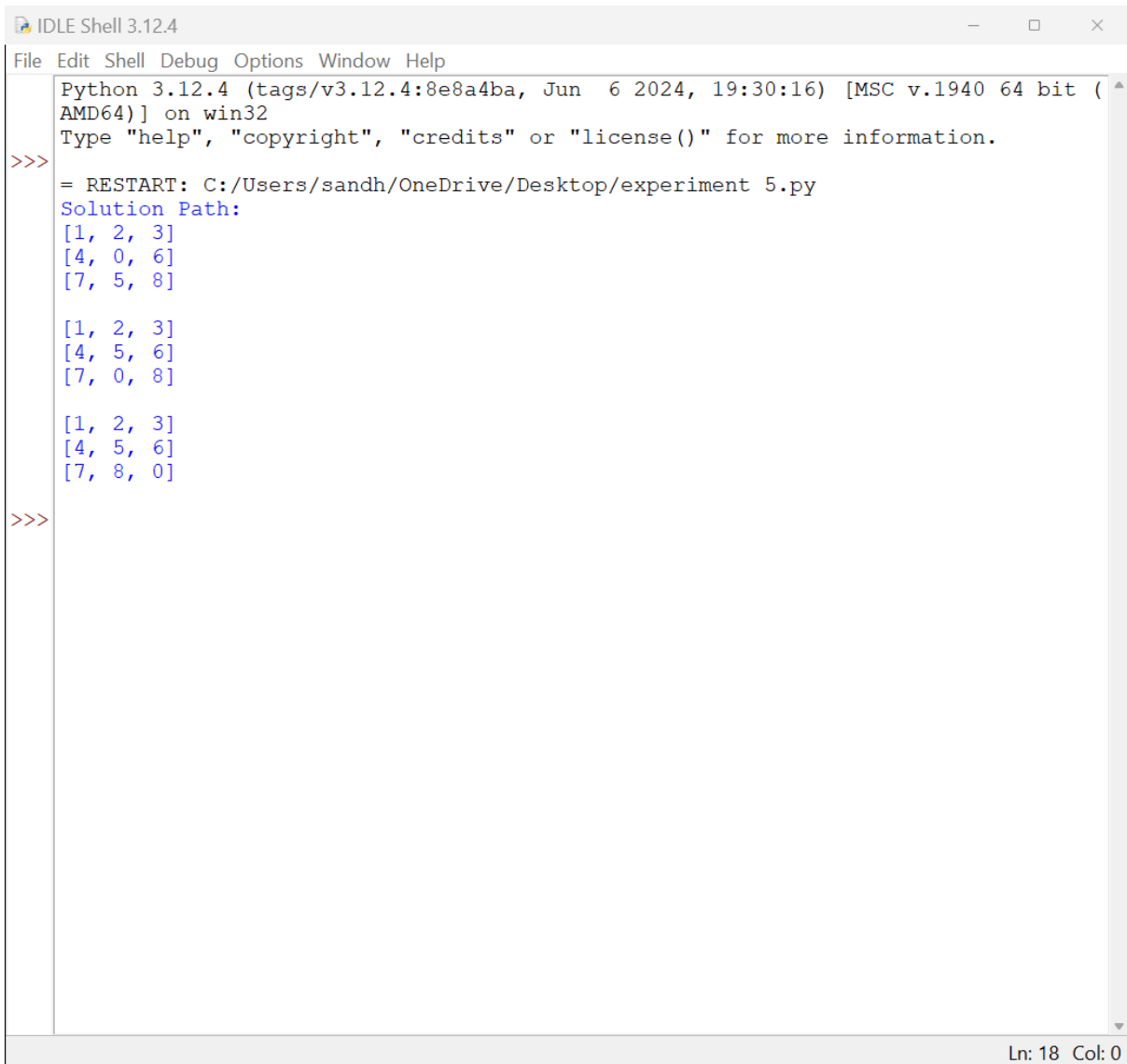
        for neighbor in get_neighbors(current):
            t = state_to_tuple(neighbor)

            if t not in visited:
                visited.add(t)
                queue.append(neighbor)

    print("No solution found.")

# Run the program
solve_puzzle(initial, goal)
```

output:



```
IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/sandh/OneDrive/Desktop/experiment 5.py
Solution Path:
[1, 2, 3]
[4, 0, 6]
[7, 5, 8]

[1, 2, 3]
[4, 5, 6]
[7, 0, 8]

[1, 2, 3]
[4, 5, 6]
[7, 8, 0]
>>>
```

Result

The Python program for the **8-Puzzle Problem** was successfully developed and executed. The program used **Breadth-First Search (BFS)** to explore possible puzzle states and successfully reached the **goal state** from the given initial state.

Conclusion

The **8-Puzzle Problem** was successfully implemented and solved using the **Breadth First Search (BFS)** algorithm. The algorithm systematically explored all possible puzzle states and found the shortest path from the initial state to the goal state. The experiment demonstrated the application of state-space search, problem-solving, and artificial intelligence techniques in finding an optimal solution efficiently.